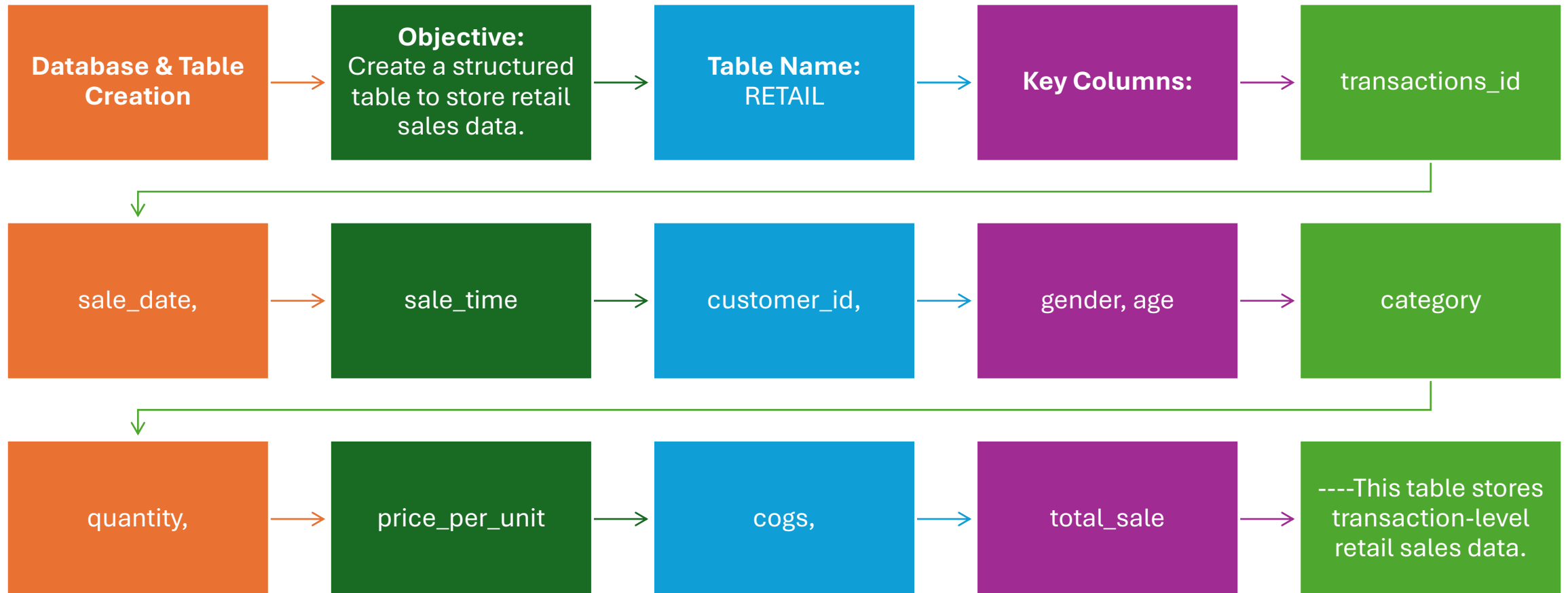


Retail Sales Data Analysis using SQL



Project Overview

This project focuses on analyzing retail sales data using SQL to extract meaningful business insights. The dataset contains transactional-level information such as sales date, time, customer details, product categories, quantity sold, and total sales. The goal of this project was to clean raw data, perform exploratory analysis, and solve real-world business problems using SQL queries.



Data Cleaning – Removing NULL Records

Objective:

Remove inaccurate or incomplete records.

Action Taken:

- Deleted rows containing NULL values in important columns.
- Ensured clean and consistent data for analysis.
- This step improves accuracy of insights.

```
--DELETE NULL VALUES
delete from RETAIL
WHERE transactions_id is null
or
  sale_date is null
or
customer_id is null
or
gender is null
or
category is null
or
  quantity is null
or
price_per_unit is null
or
  cogs is null
or
total_sale is null;
```

```
select * from RETAIL
WHERE transactions_id is null
or
  sale_date is null
or
customer_id is null
or
gender is null
or
category is null
or
  quantity is null
or
price_per_unit is null
or
  cogs is null
```

Data Insights – Basic Analysis

Key Questions Answered:

- Total number of sales transactions
- Total number of unique customers
- Total number of unique product categories

Purpose:

- Understand dataset size and structure
- Get an overview of sales activity

--DATA INSIGHT --

-- Q1. HOW MANY SALES WE HAVE

```
SELECT count(*) as total_sale FROM RETAIL;
```

-- Q2.HOW MANY CUSTOMERS WE HAVE

```
SELECT count(DISTINCT customer_id) as customer_id from RETAIL;
```

--Q3. HOW MANY UNIQUE CATEGORY WE HAVE

```
SELECT DISTINCT category as category from RETAIL;
```

Query 1 – Sales on a Specific Date

Problem Statement:

Retrieve all sales made on **05-Nov-2022**.

Insight Gained:

- Helped analyze daily sales performance
- Useful for event-based or festive sales analysis

```
68
69 --Q.1 Write a SQL query to retrieve all columns for sales made on '2022-11-05'
70
71 select * from RETAIL
72 where sale_date = TO_DATE ('2022-11-05' , 'yyyy-mm-dd');
```

Data Output Messages Notifications

Showing rows: 1 to 11 Page No: 1 of 1

	transactions_id integer	sale_date date	sale_time time without time zone	customer_id integer	gender character varying (20)	age integer	category character varying (15)	quantity integer	price_per_unit double precision	cost double precision	total_sale double precision
1	180	2022-11-05	10:47:00	117	Male	41	Clothing	3	300	129	900
2	240	2022-11-05	11:49:00	95	Female	23	Beauty	1	300	123	300
3	1256	2022-11-05	09:58:00	29	Male	23	Clothing	2	500	190	1000
4	1587	2022-11-05	20:06:00	140	Female	40	Beauty	4	300	105	1200
5	1819	2022-11-05	20:44:00	83	Female	35	Beauty	2	50	13.5	100
6	943	2022-11-05	19:29:00	90	Female	57	Clothing	4	300	318	1200
7	1896	2022-11-05	20:19:00	87	Female	30	Electronics	2	25	30.75	50
8	1137	2022-11-05	22:34:00	104	Male	46	Beauty	2	500	145	1000
9	856	2022-11-05	17:43:00	102	Male	54	Electronics	4	30	9.3	120
10	214	2022-11-05	16:31:00	53	Male	20	Beauty	2	30	8.1	60
11	1265	2022-11-05	14:35:00	86	Male	55	Clothing	3	300	111	900

Query 3 – Category-wise Sales Analysis

Problem Statement:

Calculate:

- Total sales for each category
- Total number of orders per category

Insights:

- Identifies high-performing product categories
- Helps in inventory and marketing decisions

```
81 --Q.3 Write a SQL query to calculate the total sales (total_sale) for each category.
82 select category ,
83 sum(total_sale) as total ,
84 count(*) as total_order
85 from RETAIL
86 group by 1 ;
87
```

	category character varying (15)	total double precision	total_order bigint
1	Electronics	313810	684
2	Clothing	311070	701
3	Beauty	286840	612

Query 4 – High-Value Transactions

Problem Statement:

Find transactions where total sale > 1000.

Business Impact:

- Identifies premium customers
- Useful for loyalty programs and targeted offers

```
88 -- Q.5 Write a SQL query to find all transactions where the total_sale is greater than 1000.
89 select * from RETAIL
90 where total_sale >1000;
```

	transactions_id integer	sale_date date	sale_time time without time zone	customer_id integer	gender character varying (20)	age integer	category character varying (15)	quantity integer	price_per_unit double precision	cogs double precision	total_sale double precision
1	522	2022-07-09	11:00:00	52	Male	46	Beauty	3	500	145	1500
2	559	2022-12-12	10:48:00	5	Female	40	Clothing	4	300	84	1200
3	1522	2022-11-14	08:35:00	48	Male	46	Beauty	3	500	235	1500
4	1559	2022-08-20	07:40:00	49	Female	40	Clothing	4	300	144	1200
5	421	2022-04-08	08:43:00	66	Female	37	Clothing	3	500	235	1500
6	1421	2022-01-17	07:07:00	59	Female	37	Clothing	3	500	185	1500
7	484	2022-03-13	07:52:00	135	Female	19	Clothing	4	300	75	1200
8	1484	2022-11-23	09:29:00	22	Female	19	Clothing	4	300	147	1200
9	15	2022-07-01	11:50:00	75	Female	42	Electronics	4	500	210	2000
10	743	2022-08-07	07:54:00	55	Female	34	Beauty	4	500	260	2000
11	1015	2022-03-09	11:53:00	94	Female	42	Electronics	4	500	200	2000
12	1743	2022-10-26	09:37:00	47	Female	34	Beauty	4	500	250	2000
13	742	2022-03-19	06:08:00	37	Female	38	Electronics	4	500	195	2000
14	1742	2022-11-22	08:25:00	18	Female	38	Electronics	4	500	220	2000
15	420	2022-01-02	10:53:00	28	Female	22	Clothing	4	500	200	2000
16	1420	2022-04-15	07:01:00	138	Female	22	Clothing	4	500	205	2000
17	592	2022-12-26	09:15:00	77	Female	46	Beauty	4	500	275	2000
18	1592	2022-03-16	09:08:00	81	Female	46	Beauty	4	500	155	2000
19	720	2022-04-08	08:50:00	116	Female	56	Beauty	3	500	235	1500
20	1720	2022-10-10	08:51:00	28	Female	56	Beauty	3	500	190	1500

Total rows: 306 Query complete 00:00:00.273 CRLF Ln 97, Col 12

Query 5 – Gender-wise Transactions

Problem Statement:

Find total transactions made by:

- Each gender
- In each product category

Insights:

- Understand customer demographics
- Helps in gender-focused marketing strategies

```
92 -- Q.6 Write a SQL query to find the total number of transactions (transaction_id) made by each gender in each category.
93 select category ,gender ,
94 count(*) transaction_id
95 from RETAIL
96 GROUP BY category,gender
97 order by 1;
98
```

	category character varying (15)	gender character varying (20)	transaction_id bigint
1	Beauty	Female	330
2	Beauty	Male	282
3	Clothing	Female	347
4	Clothing	Male	354
5	Electronics	Male	344
6	Electronics	Female	340

Advanced Analysis – Best Selling Month per Year

Problem Statement:

- Calculate average monthly sales
- Identify best-selling month for each year

SQL Concepts Used:

- Date extraction
- Window function (RANK)

Business Value:

- Seasonal sales trend analysis
- Better planning for promotions

```
98
99 --Q.7 Write a SQL query to calculate the average sale for each month. Find out best selling month in each year
100 select year,month,avg_sale
101 from
102 (select
103 extract(year from sale_date) as year,
104 extract (month from sale_date) as month,
105 avg(total_sale) as avg_sale,
106 rank() over (partition by extract(year from sale_date) order by avg(total_sale)desc) as rank
107 from RETAIL
108 GROUP BY 1,2
109 )AS T1
110 WHERE RANK =1;
...
```

Data Output Messages Notifications

Showing rows: 1 to 2 Page 1

	year numeric	month numeric	avg_sale double precision
1	2022	7	541.3414634146342
2	2023	2	535.531914893617

Advanced Analysis – Top 5 Customers

Problem Statement:

Identify top 5 customers based on highest total sales.

Insights:

- Recognizes high-value customers
- Supports customer retention strategies

```
111
112 --Q.8 Write a SQL query to find the top 5 customers based on the highest total sales
113 select customer_id,
114 sum(total_sale)as top_sale
115 from RETAIL
116 group by 1
117 order by 2 DESC
118 limit 5;
119
```

Data Output Messages Notifications

	customer_id integer	top_sale double precision
1	3	38440
2	1	30750
3	5	30405
4	2	25295
5	4	23580

Time-Based Analysis – Shift-wise Orders

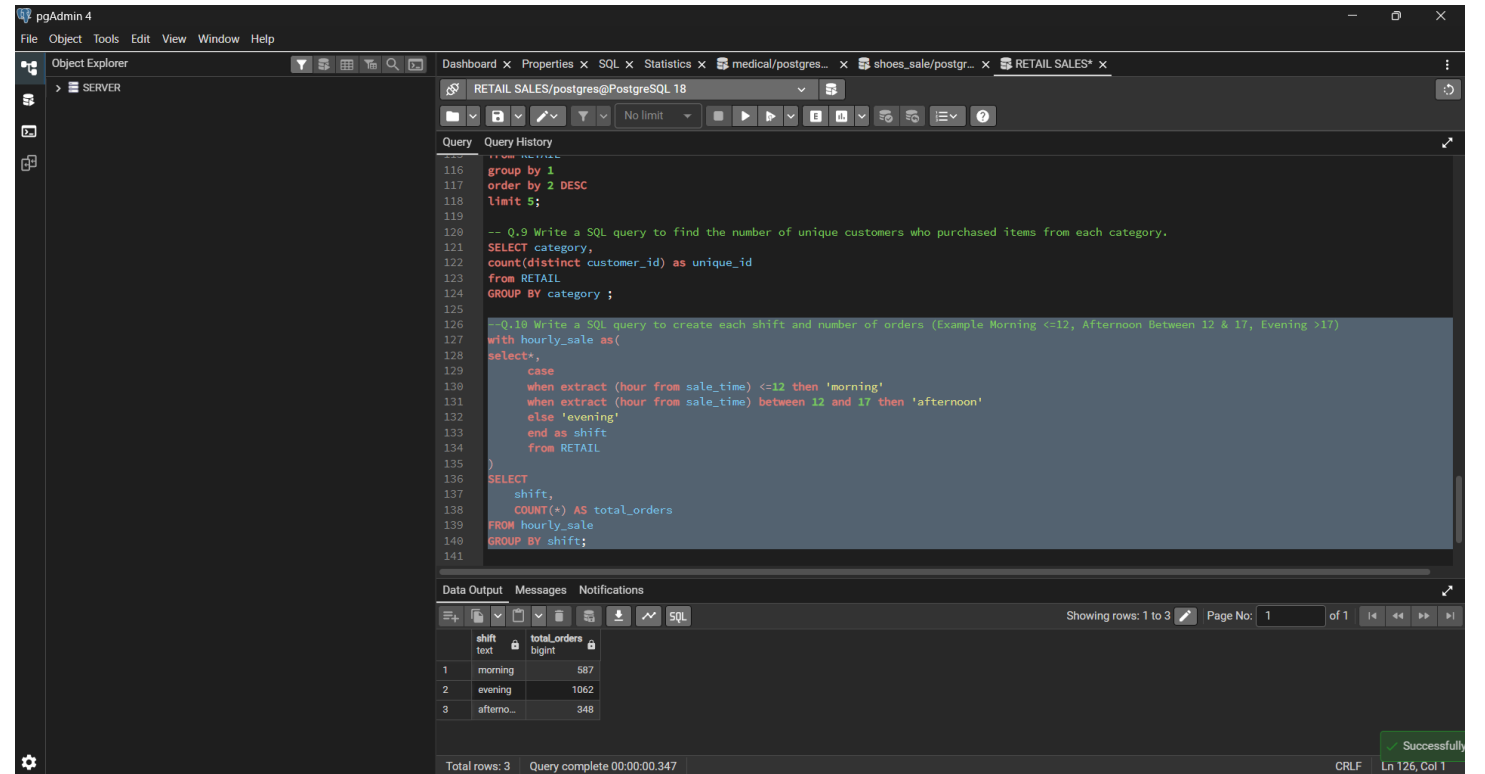
Problem Statement:

Divide sales into shifts:

- Morning (≤ 12)
- Afternoon (12–17)
- Evening (>17)

Insights:

- Identifies peak order time
- Helps in staff and resource planning



The screenshot shows the pgAdmin 4 interface with the following components:

- Object Explorer:** Shows the 'SERVER' tree on the left.
- Query Editor:** Contains two SQL queries. The first query is a simple group by query. The second query is a more complex one that creates a CTE named 'hourly_sale' and then selects the shift and total orders.
- Data Output:** Displays the results of the second query in a table with 3 rows and 2 columns: 'shift' and 'totalOrders'.
- Messages:** Shows a 'Successfully' message at the bottom right.

```
116 group by 1
117 order by 2 DESC
118 limit 5;
119
120 -- Q.9 Write a SQL query to find the number of unique customers who purchased items from each category.
121 SELECT category,
122 count(distinct customer_id) as unique_id
123 from RETAIL
124 GROUP BY category ;
125
126 --Q.10 Write a SQL query to create each shift and number of orders (Example Morning <=12, Afternoon Between 12 & 17, Evening >17)
127 with hourly_sale as(
128 select*,
129 case
130 when extract (hour from sale_time) <=12 then 'morning'
131 when extract (hour from sale_time) between 12 and 17 then 'afternoon'
132 else 'evening'
133 end as shift
134 from RETAIL
135 )
136 SELECT
137     shift,
138     COUNT(*) AS total_orders
139 FROM hourly_sale
140 GROUP BY shift;
141
```

shift	totalOrders
1 morning	587
2 evening	1062
3 afterno...	348

Total rows: 3 Query complete 00:00:00.347



Conclusion & Learnings

Clean data is crucial for accurate analysis

SQL can solve real-world business problems

Gained hands-on experience with:

- Data cleaning
- Business analysis
- Advanced SQL queries

This project strengthens my Data Analyst portfolio.